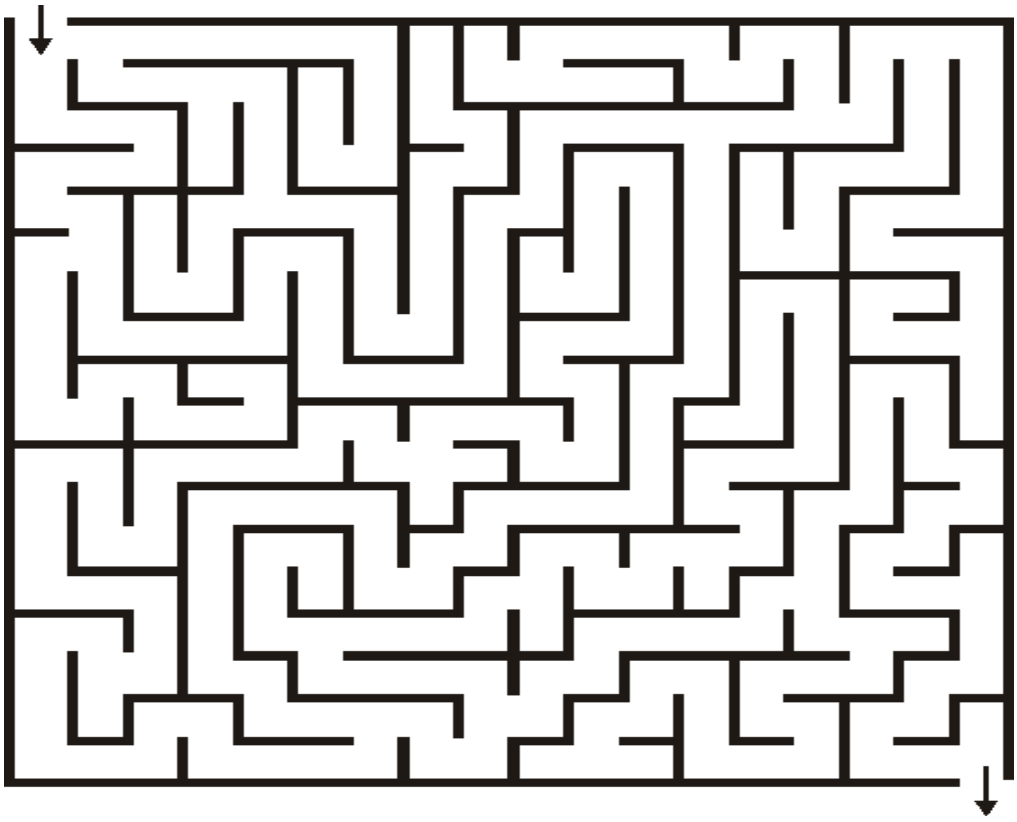
Building and Running mazes

The task of finding a path through a maze seems forever fascinating. You can generate an elaborate maze on a computer and use graphics to watch it be traversed. **The goal is to find a path from the opening at the left edge to the opening at the right edge**. Although you can traverse the maze manually by trial and error, it is more interesting to develop an algorithm to do it automatically.

Write and execute a program that
(a) generates and displays a rectangular maze of R rows and C columns and
(b) finds (and displays) the path through the maze from start to end. The program generates mazes randomly, but they must be **proper**; that is, every one of the R by C cells is connected by a unique, albeit tortuous, path to every other cell.

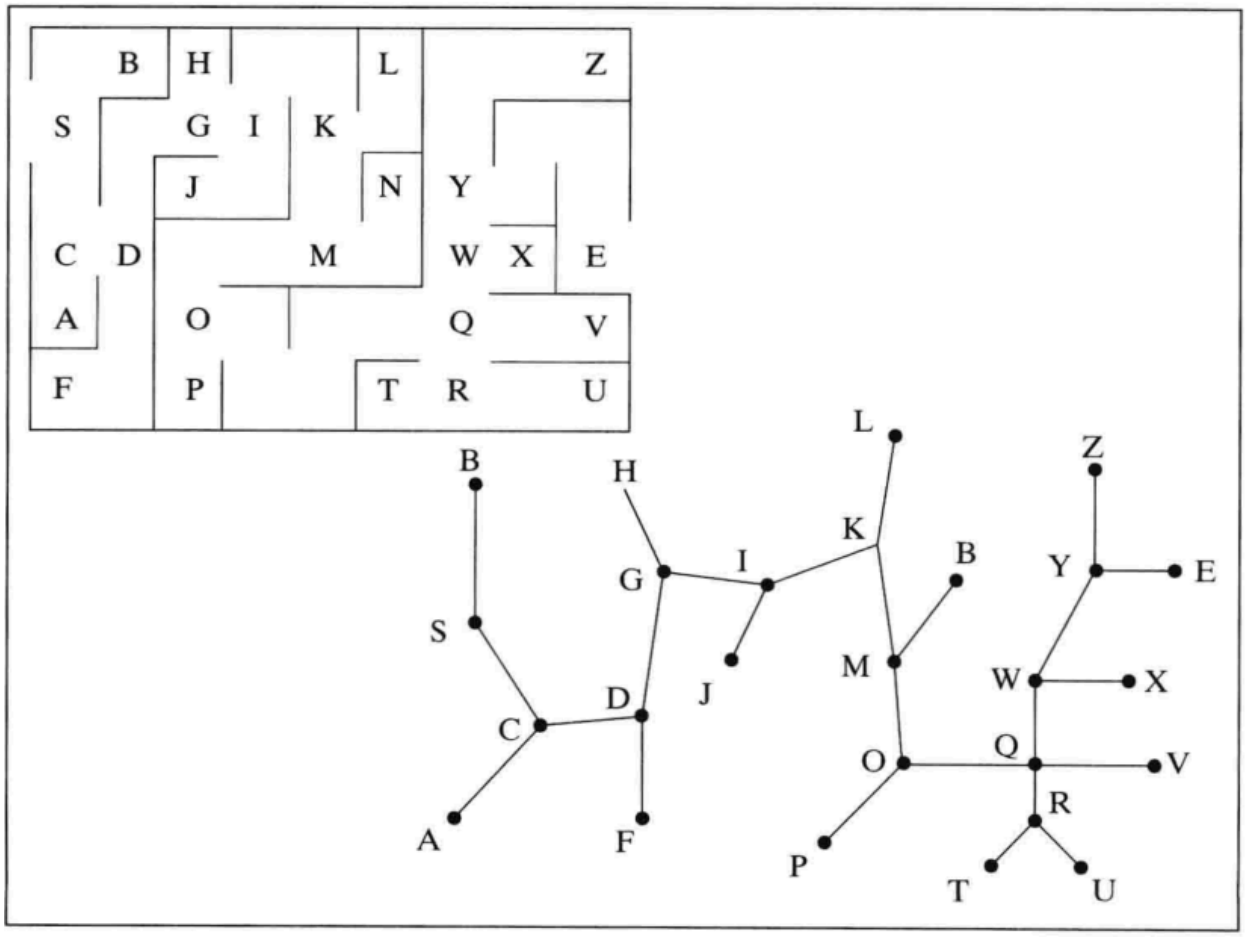


How should a maze be represented? One way is to state, for each cell, whether its north wall is intact and its east wall is intact, suggesting the following data structure:

```
char northWall[R][C], eastWall[R][C];
```

Logic and Structure

- **Wall Integrity:** If northwall[i][j] is **1**, the ij^{th} cell has a solid upper wall; otherwise, the wall is missing.
- **The Bottom Edge:** The zeroth row is a **phantom row** of cells below the maze whose north walls make up the bottom edge of the maze.
- **The Left Edge:** Similarly, eastwall[i][0] specifies where any gaps appear in the left edge of the maze.



Generating a Maze

Start with all walls intact, so that the maze is a simple grid of horizontal and vertical lines. The program draws this grid. An invisible **“mouse”** whose job is to **“eat”** through walls to connect adjacent cells is initially placed in some arbitrarily chosen cell.

The mouse checks the four neighbor cells (above, below, left, and right) and, for each neighbor, asks whether it has all four walls intact. If not, the cell has previously been visited and so is already on some path. The mouse may detect several candidate cells that haven't been visited:

1. It chooses one randomly and eats through the connecting wall, saving the locations of the other candidates on a **stack**.
2. The wall that is eaten is erased, and the mouse repeats the process.
3. When it becomes trapped in a dead end—surrounded by visited cells—it pops an unvisited cell and continues.
4. When the stack is empty, all cells in the maze have been visited.

A **"start"** and **"end"** cell is then chosen randomly, most likely along some edge of the maze. It is delightful to watch the maze being formed dynamically as the mouse eats through walls.

> **Question:** Might a queue be better than a stack for storing candidates? How would it affect the order in which later paths are created?

Running the Maze

To run the maze, we use a **"backtracking"** algorithm. At each step, the mouse tries to move in a random direction. If there is no wall, it places its position on a stack and moves to the next cell. The cell that the mouse is in can be drawn with a **red dot**. When the mouse runs into a dead end, it can change the color of the cell to **blue** and backtrack by popping the stack. The mouse can even put a wall up to avoid ever trying the dead-end cell again.

Addendum

Proper mazes aren't too challenging, because you can always traverse them using the **"shoulder-to-the-wall"** rule, wherein you trace the maze by rubbing your shoulder along the left-hand wall. At a dead end, you sweep around and retrace the path, always maintaining contact with the wall.

Because the maze is a **"tree,"** you will ultimately reach your destination. In fact, there can even be cycles in the graph and you will still always find the end, as long as both the starting and ending cells are on outer boundaries of the maze. **(Why?)** To make things more interesting, place the starting and ending cells in the interior of the maze, and also let the mouse eat some extra

walls (maybe randomly 1 in 20 times). In this way, some cycles may be formed that encircle the ending cell and defeat the “shoulder-to-the-wall” method.

Submission

1. The GitHub Repository

- **README.md:** This is the "face" of your project. Include a brief description of how your maze generator works (mentioning the stack-based DFS "mouse" logic).
- **Code Organization:** Ensure your northWall and eastWall arrays are clearly defined and commented on as per the assignment's data structure requirements.
- **Clear Commit History:** It shows the evolution of your work (e.g., "Initial grid setup," "Implemented mouse movement," "Added backtracking").

2. The Loom Recording

- **The "Eating" Process:** Show the maze being generated dynamically. It’s the most "delightful" part, as the text mentioned, so don't skip it!
- **The Solver:** Demonstrate the "red dot" mouse finding its way and the "blue dots" marking dead ends.

3. The "Challenge" (Bonus Points?)

If you want to go the extra mile based on that **Addendum**, show a version where the mouse eats an extra wall (1 in 20) to create a cycle. Demonstrating how that breaks the "shoulder-to-the-wall" rule would definitely impress whoever is grading this.